

MongoDB CRUD Operations

Each question is shown in the blue box, followed by its answer. Commands are for mongosh and work the same in the older mongo shell. ObjectIds in the outputs are examples and will differ on your system.

Question 1

Question 1. Create a MongoDB collection named 'restaurants' and use `insertOne` to add a new restaurant document with fields: `name`, `cuisine`, `location`, and `rating`.

Answer: switch to a database (this example uses `foodieApp`), create the collection, and insert one document.

```
use foodieApp

db.createCollection("restaurants")

db.restaurants.insertOne({
  name: "Burger Hub",
  cuisine: "Fast Food",
  location: { city: "Mumbai", area: "Andheri" },
  rating: 4.2
})
```

Output:

```
switched to db foodieApp
{ ok: 1 }
{
  acknowledged: true,
  insertedId: ObjectId('66f3c1a2e4b0a1b2c3d4e701')
}
```

Creating the collection first is optional. `insertOne()` would create it automatically if it did not exist. This document (Burger Hub) is also the one updated in Question 4.

Question 2

Question 2. Use `insertMany` to add 3 new documents to the 'movies' collection, each representing a different movie with fields: `title`, `genre`, `releaseYear`, and `imdbRating`. Hint: Think of movies currently trending on BookMyShow or Netflix.

Answer: `insertMany()` takes an array of documents and inserts them all in one command.

```
db.movies.insertMany([
  { title: "Inception",    genre: "Sci-Fi",    releaseYear: 2010, imdbRating: 8.8 },
  { title: "Dangal",       genre: "Biography", releaseYear: 2016, imdbRating: 8.3 },
  { title: "Oppenheimer",  genre: "Drama",    releaseYear: 2023, imdbRating: 8.3 }
])
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('66f3c1b0e4b0a1b2c3d4e702'),
    '1': ObjectId('66f3c1b0e4b0a1b2c3d4e703'),
    '2': ObjectId('66f3c1b0e4b0a1b2c3d4e704')
  }
}
```

Note: these are well-known, popular films used as examples, not a live trending list. For your submission you can replace them with whatever is trending on BookMyShow or Netflix when you do the task. The IMDb ratings shown are approximate and change over time, so check the current values on IMDb.

Question 3

Question 3. Write a query using `find` to display all products from a `'flipkart_products'` collection where the price is less than 1000.

Answer: use the `$lt` (less than) comparison operator.

```
db.flipkart_products.find({ price: { $lt: 1000 } })
```

Sample data to test with:

```
db.flipkart_products.insertMany([
  { name: "Bluetooth Speaker", category: "Electronics", price: 799 },
  { name: "Running Shoes",      category: "Footwear",    price: 1499 },
  { name: "Cotton T-Shirt",     category: "Clothing",   price: 499 },
  { name: "Laptop Bag",         category: "Accessories", price: 999 },
  { name: "Smartwatch",         category: "Electronics", price: 2499 }
])
```

Output:

```
[
  { _id: ObjectId('66f3c1c5e4b0a1b2c3d4e705'), name: 'Bluetooth Speaker', category: 'Electronics', price: 799 },
  { _id: ObjectId('66f3c1c5e4b0a1b2c3d4e707'), name: 'Cotton T-Shirt', category: 'Clothing', price: 499 },
  { _id: ObjectId('66f3c1c5e4b0a1b2c3d4e708'), name: 'Laptop Bag', category: 'Accessories', price: 999 }
]
```

Points to remember:

- `$lt` means strictly less than, so a product priced exactly 1000 is not included. Use `$lte` to include it.
- The price must be stored as a number. If it is stored as a string like "799", the comparison will not work as expected.
- To show only the name and price, add a projection: `db.flipkart_products.find({ price: { $lt: 1000 } }, { name: 1, price: 1, _id: 0 })`.

Question 4

Question 4. Update the rating of a restaurant named 'Burger Hub' in the 'restaurants' collection to 4.7 using `updateOne`.

Answer: the first argument is the filter and the second uses the `$set` operator to change only the rating field.

```
db.restaurants.updateOne(
  { name: "Burger Hub" },
  { $set: { rating: 4.7 } }
)
```

Output:

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

`matchedCount: 1` means one document matched the filter, and `modifiedCount: 1` means it was changed. Verify the result:

```
db.restaurants.findOne({ name: "Burger Hub" })

{
  _id: ObjectId('66f3c1a2e4b0a1b2c3d4e701'),
  name: 'Burger Hub',
  rating: 4.7
}
```

```

    cuisine: 'Fast Food',
    location: { city: 'Mumbai', area: 'Andheri' },
    rating: 4.7
}

```

Points to remember: `updateOne()` changes only the first matching document. If several restaurants are named "Burger Hub", use `updateMany()` to update all of them. Without `$set`, MongoDB gives an error because `updateOne()` requires an update operator.

Question 5

Question 5. Delete all orders from an 'orders' collection where the status is 'cancelled' using `deleteMany`. Constraint: Only delete documents with status exactly equal to 'cancelled'.

Answer: filter on status with an exact match. A plain `{ status: "cancelled" }` filter is an exact equality check.

```
db.orders.deleteMany({ status: "cancelled" })
```

Sample data to test with:

```

db.orders.insertMany([
  { order_id: 1, item: "Laptop",      status: "delivered" },
  { order_id: 2, item: "Headphones", status: "cancelled" },
  { order_id: 3, item: "Shoes",       status: "shipped" },
  { order_id: 4, item: "Watch",       status: "cancelled" },
  { order_id: 5, item: "Book",        status: "Cancelled" }
])

```

Output:

```
{ acknowledged: true, deletedCount: 2 }
```

Verify what is left:

```

db.orders.find({}, { _id: 0 })

[
  { order_id: 1, item: 'Laptop', status: 'delivered' },
  { order_id: 3, item: 'Shoes', status: 'shipped' },
  { order_id: 5, item: 'Book', status: 'Cancelled' }
]

```

Only orders 2 and 4 were deleted. Order 5 has "Cancelled" with a capital C, so it is not exactly equal to "cancelled" and is correctly kept. This follows the constraint.

Safety tips:

- Preview first with `db.orders.countDocuments({ status: "cancelled" })` or `db.orders.find({ status: "cancelled" })` to see what will be deleted.
- Do not use regular expressions or operators like `$ne` here, because they can match more documents than intended.
- Never run `db.orders.deleteMany({})`, because an empty filter deletes every document in the collection.